Generated from `docs/handoff-guides/markdown/migration-schema.md` by `tools/guide-publisher`.

Migrations Handoff Guide

Use this guide when you need to describe **how a Blueprint model changes over time**: what is being added, removed, renamed, redirected, split, merged, or updated, and why.

This guide is about **model evolution**, not only release notes.

KNOWLEDGE AREA

This guide helps with **planned model changes over time**. Technically, it covers **model evolution and change traceability**.

Guide family map

Where this guide sits in the full Blueprint Handoff Atlas family.

ROOT & CROSS-CUTTING

Handoff Atlas | Blueprint Bundle | Metamodel Vocabulary | **Migrations**

DESIGN

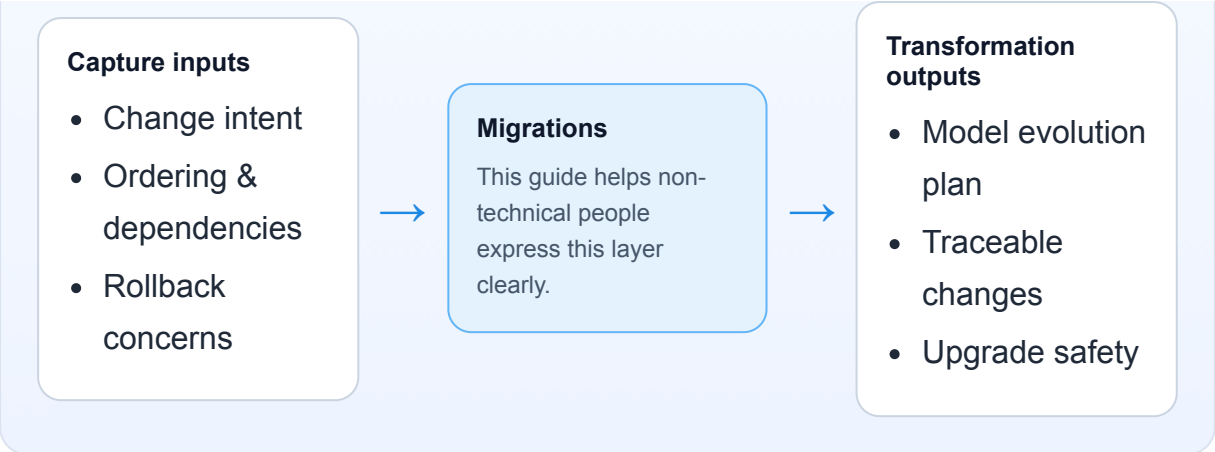
Architecture | Concepts | Domain | Dynamics | Infrastructure | Interactions | Models | Quality | Rules | Story

GOVERNANCE

Capability | Decisions | Motivation | Organization | Roadmap | Leverage | Test Cases | Value Stream

Capture → focus → transformation

What to gather first, what this layer focuses on, and what a modeler or AI agent can produce from it.



Main entities in this guide

- **Migration** - the named change package with identity, status, rationale, and dependency information.
- **Entity Change** - a change to a whole modeled thing, such as add, remove, split, or merge.
- **Property Change** - a change to one field or attribute on an existing thing.
- **Relationship Change** - a change to links between modeled things.
- **Meta Change** - a cross-cutting change such as tags, file moves, or bundle-level conventions.
- **Rollback** - the note about whether and how the change can be reversed safely.
    - Often confused with: a roadmap plan. A migration describes the model change itself, not the broader delivery schedule.

What belongs here

- what change is being made to the Blueprint model
- why the change is needed
- what parts of the model are affected
- what order changes must happen in
- whether rollback or dependency concerns exist
- what decisions or rationale motivated the migration

What does not belong here

- generic release notes with no model impact
- low-level code migration scripts by themselves
- broad strategic goals without a specific model change

Core things to capture

- the migration name and purpose
- what model entities or relationships are changing
- why the change is needed
- whether the change is breaking
- what dependencies or sequencing matter
- whether rollback is easy, risky, or impossible

Core relationships to capture

- migration -> changed entities/properties/relationships
- migration -> prior migration it depends on
- migration -> related decisions or rationale
- migration -> rollback risk or external follow-up action

Modus operandi

Describe migrations as explicit change packages:

1. state what is changing
2. explain why it is changing
3. group the affected entities, properties, and relationships
4. state ordering and dependencies
5. state rollback feasibility and risks
6. mention related decisions when relevant

Prompt set

- what changed in the Blueprint model?
- why is the change needed now?
- what entities, properties, or relationships are affected?
- is the change breaking?
- what must happen before this migration?
- what would make rollback difficult?

Free-text intake template

- **Migration name:**
- **Purpose:**
- **Status:**
- **What changes:**
    - entities:

    - properties:
    - relationships:
    - meta/bundle changes:
- **Why:**
- **Depends on:**
- **Breaking or non-breaking:**
- **Rollback notes:**
- **Related decisions:**

Worked example

- **Migration name:** split-checkout-and-post-purchase-slices
- **Purpose:** separate one commerce slice into two clearer business slices
- **Status:** pending
- **What changes:** move concepts and stories into checkout and post-purchase; redirect references; add shared root architecture notes
- **Why:** the old model mixed two domains with different ownership and change rates
- **Depends on:** establish new slice names first
- **Breaking or non-breaking:** potentially breaking for downstream generators and references
- **Rollback notes:** rollback is possible only if moved entities keep their old identity trace

Layer-specific handoff guidance

When handing off migration notes, the most important thing is **change clarity**.

A good handoff should make it obvious:

- what is changing
- why it is changing
- what must happen first
- what makes rollback easy or hard

Do not send only release-style summaries. The receiver needs concrete affected entities, relationships, dependencies, and risks.

How this becomes YAML

A technical modeler or AI agent can transform this into migration metadata plus ordered entity, property, relationship, and meta changes with dependency and rollback information.

Common mistakes

- writing release notes instead of model change intent
- failing to name the affected entities or relationships
- omitting why the migration exists
- forgetting dependency order or rollback concerns